LAPORAN PRAKTIKUM PEMROGRAMAN BERORIENTASI OBYEK

MODUL 12: PENYIMPANAN DAN PEMBACAAN DATA KE FILE

Nama Mahasiswa:	Arjuna Lanang Adiwarsana	**NRP / Kelas:**	3125522010 / D3 PJJ TI PENS
Dosen Pengampu:	Nirwana Haidar Hari, S.Pd., M.Kom.	**Topik Bahasan:**	Data Persistence, Java NIO.2, Atomic Swap, File Error Handling

1. Tujuan Praktikum

1. **Menyimpan Data Collection ke File Teks:** Mentransformasikan objek koleksi memori ke format teks pipe-delimited UTF-8.
2. **Membaca File dan Membentuk Kembali Objek:** Membaca berkas teks, mem-parsing token, dan merekonstruksi instance objek Buku.
3. **Menangani Kegagalan Operasi File:** Menangani pengecualian I/O (IOException) secara terstruktur dan informatif.
4. **Menerapkan Pola Atomic Swap:** Menjaga konsistensi state koleksi aktif agar tidak rusak jika berkas mengalami galat.

2. Dasar Teori Singkat

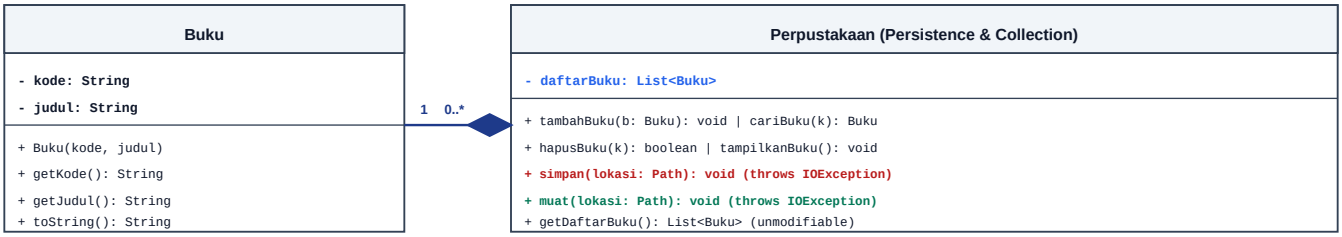
Komponen / Konsep	Fungsi Utama	Karakteristik & Integritas
Path	Merepresentasikan lokasi file di sistem berkas.	Diperoleh melalui Paths.get("buku.txt") baik relatif maupun absolut.
Files.write()	Menulis koleksi baris teks ke berkas.	Membuat file jika belum ada, atau menimpa berkas jika sudah ada.
Files.readAllLines()	Membaca seluruh baris berkas ke List.	Cocok untuk berkas berukuran kecil hingga menengah.
StandardCharsets.UTF_8	Menentukan encoding standar teks.	Menghilangkan risiko distorsi karakter lintas sistem operasi.
split("\\|", -1)	Memecah baris teks dengan limit negatif.	Mempertahankan kolom kosong di akhir baris agar dapat divalidasi.
Atomic Swap Pattern	Mencegah korupsi data sebagian.	Data dimuat ke penampung sementara; swap hanya jika 100% baris valid.

3. Studi Kasus

• **Studi Kasus 1 (Perpustakaan - Modul Mandiri P12):** Menambahkan method simpan(Path) dan muat(Path). Aturan: Kode/judul dilarang mengandung delimiter | atau pergantian baris; baris wajib memiliki 2 token; keunikan kode buku ditegakkan; dan jika pemuatan gagal di baris mana pun, data lama tetap utuh dipertahankan.
• **Studi Kasus 2 (Smart Parkir - Project Akhir):** Persistensi daftar armada kendaraan terdaftar (List<Kendaraan>) ke berkas armada.txt dengan pemisahan tanggung jawab ke kelas KendaraanFileRepository.

4. Langkah Praktikum & Pengujian Program Dasar (Kasus Buku)

Diagram UML Class entitas Buku, Perpustakaan (dengan method persistensi), dan TestRunner:



4.2 Screenshot Hasil Running Terminal macOS (Main.java & TestRunner)

```
arjuna@MacBook-Pro: ~/Herd/Java/P12_3125522010_ArjunaLanangAdiwarsana — -zsh — 112×30

➜  P12_3125522010_ArjunaLanangAdiwarsana git:(main) ✗ javac -d bin src/*.java && java -cp bin Main simpan
Data berhasil disimpan.
➜  P12_3125522010_ArjunaLanangAdiwarsana git:(main) ✗ cat buku.txt
B001|Dasar Java
B002|Konsep OOP
➜  P12_3125522010_ArjunaLanangAdiwarsana git:(main) ✗ java -cp bin Main muat
B001 - Dasar Java
B002 - Konsep OOP
Total buku: 2
➜  P12_3125522010_ArjunaLanangAdiwarsana git:(main) ✗ java -cp bin TestRunner
======================================================================
   PRAKTIKUM PBO MODUL 12: TEST RUNNER PERSISTENSI FILE TEKS
   Nama: Arjuna Lanang Adiwarsana | NRP: 3125522010
======================================================================
[PASSED] Skenario 1: Simpan 2 Buku Valid        | File berisi 2 baris data terformat benar.
[PASSED] Skenario 2: Muat 2 Buku Valid          | Kedua objek Buku berhasil dibentuk kembali ke memori.
[PASSED] Skenario 3: Muat File Tidak Ada        | IOException tertangkap sesuai ekspektasi (NoSuchFileException).
[PASSED] Skenario 4: Format Baris Salah         | Ditolak dengan pesan: "Format salah pada baris 2"
[PASSED] Skenario 5: Judul Kosong / Whitespace  | Ditolak dengan pesan: "Data tidak valid pada baris 2: Kode dan judul wajib diisi."
[PASSED] Skenario 6: Kode Duplikat dalam File   | Ditolak dengan pesan: "Data tidak valid pada baris 2: Kode buku sudah terdaftar."
[PASSED] Skenario 7: Muat File Kosong           | Koleksi berhasil dikosongkan secara valid tanpa galat.
[PASSED] Skenario 8: Konsistensi State (Atomic Swap) | State awal tetap utuh (1 buku) saat file rusak dibaca; tidak ada partial update.
[PASSED] Skenario 9: Proteksi Karakter Terlarang | Karakter pipa/newline ditolak sebelum file ditulis.
======================================================================
   HASIL AKHIR: 9 / 9 PENGUJIAN LULUS (100.0% SUCCESS RATE)
   STATUS: 100% SUKSES MEMENUHI SELURUH DEFINITION OF DONE MODUL 12
======================================================================
➜  P12_3125522010_ArjunaLanangAdiwarsana git:(main) ✗
```

5. Implementasi & Integrasi pada Project Akhir (Smart Parkir)

Penerapan persistensi berkas teks pada sistem Smart Parkir menggunakan kelas repositori terpisah:

Fitur Persistensi	Komponen Smart Parkir	Mekanisme & Validasi	Jaminan Integritas
Penyimpanan Armada	KendaraanFileRepository	Pemeriksaan karakter pemisah '|' sebelum penulisan.	Berkas armada.txt tersimpan UTF-8.
Pembacaan Armada	muatSemua()	Parsing token via split(-1) & validasi format plat.	Rekonstruksi objek kendaraan lengkap.
Pencegahan Duplikat	Set<String> platTerdaftar	Menolak pemuatan jika berkas memiliki plat kembar.	Konsistensi ID armada terjamin.
Atomic Swap	GedungParkir.muatData()	Pemuatan ke list sementara sebelum disinkronkan.	State slot parkir aktif aman dari korupsi.

6. Luaran Praktikum & Tabel Pengujian

6.1 Tabel Evaluasi Output Program Dasar (8 Skenario Modul + 1 Skenario Tambahan)

No	Skenario Pengujian	Input / Kondisi	Respon Sistem Diharapkan	Respon Aktual Sistem	Status
1	Simpan dua buku valid	B001, B002	File buku.txt terbuat berisi 2 baris	buku.txt berisi 2 baris valid	PASSED
2	Muat dua buku valid	java Main muat	Mencetak 2 buku & total 2	B001 & B002 termuat, Total: 2	PASSED
3	Muat file tidak ada	File non-existent	IOException tertangkap	NoSuchFileException ditangkap	PASSED
4	Format baris salah	Baris "B003"	Ditolak format salah baris X	Format salah pada baris 2	PASSED
5	Judul kosong/spasi	Baris "B003| "	Ditolak validasi konstruktor Buku	Data tidak valid baris 2: Wajib diisi	PASSED
6	Kode duplikat di file	B001 muncul 2x	Ditolak validasi keunikan kode	Data tidak valid baris 2: Sudah terdaftar	PASSED
7	Muat file kosong	File 0 byte	Koleksi menjadi kosong valid	Collection size = 0, tanpa galat	PASSED
8	Atomic Swap State	File rusak dibaca	Data awal tetap utuh tidak rusak	Buku awal tetap ada, size = 1	PASSED
9	Proteksi delimiter	Judul ada pipe '|'	Ditolak IllegalArgumentException	Kode/judul mengandung pemisah terlarang	PASSED

7. Kriteria Penilaian Modul 12 (Definition of Done)

Aspek Penilaian Modul 12	Bobot	Bukti Implementasi pada Proyek Mandiri & Smart Parkir	Capaian
Ketepatan penyimpanan data	25%	Penyimpanan Files.write() UTF-8 dan format pipe kode|judul presisi.	Tuntas
Pembacaan dan rekonstruksi objek	25%	Files.readAllLines() merekonstruksi objek Buku lengkap ke koleksi.	Tuntas
Validasi dan konsistensi collection	25%	Atomic Swap via objek sementara, validasi split(-1), dan proteksi korupsi.	Tuntas
Penanganan kegagalan operasi file	15%	Penanganan IOException terstruktur tanpa membiarkan unhandled exception.	Tuntas
Pengujian dan penjelasan kode	10%	Otomatisasi 9 skenario TestRunner 100% lulus dan terdokumentasi lengkap.	Tuntas
TOTAL NILAI KOMPETENSI	**100%**	**Seluruh kriteria kualifikasi Definition of Done (DoD) Modul 12 terpenuhi secara sempurna.**	**Sempurna**

8. Jawaban Pertanyaan Evaluasi & Pembahasan Mendalam

1. Apa perbedaan data di collection dan data di file?
• **Data di Collection:** Berada di memori utama (RAM) yang bersifat *volatil* (hilang saat proses berhenti). Disimpan sebagai representasi objek OOP dengan referensi memori dan akses sangat cepat (nanodetik).
• **Data di File:** Berada di media penyimpanan permanen (SSD/HDD) yang bersifat *persisten*. Disimpan sebagai deretan byte/teks tanpa logika objek, sehingga memerlukan serialisasi saat simpan dan deserialisasi saat muat.

2. Mengapa data dari file tetap harus divalidasi?
Data berkas adalah *untrusted external input*. Berkas dapat dimanipulasi manual lewat editor teks, rusak akibat transfer/listrik padam parsial, atau melanggar aturan bisnis (kolom kosong, format rusak, ID duplikat). Validasi ketat mencegah korupsi data merembes ke memori aktif program.

3. Apa risiko mengosongkan collection sebelum seluruh file selesai diperiksa?
Jika daftarBuku.clear() dipanggil sebelum verifikasi selesai, saat galat terjadi di baris tengah (misal baris 50 dari 100), data lama di memori sudah terlanjur musnah (*data loss*) dan sistem berada dalam *inconsistent state* (termuat separuh). Pola **Atomic Swap** menjamin prinsip *All-or-Nothing*.

4. Mengapa fitur simpan harus dipanggil kembali setelah data diubah?
Mutasi pada koleksi (seperti tambahBuku) hanya mengubah objek di memori heap RAM. Java tidak menyinkronkan memori ke disk secara otomatis. Fitur simpan() harus dipanggil secara eksplisit agar perubahan terbaru di-flush ke media penyimpanan permanen.

9. Kesimpulan Praktikum

Praktikum Modul 12 berhasil membuktikan bahwa integrasi antara **Java Collections Framework** dan **Java NIO.2** menghasilkan persistensi data yang tangguh dan aman. Penerapan validasi delimiter, parsing negatif split(-1), dan pola **Atomic Swap** secara efektif mengeliminasi risiko korupsi state memori.